Mecha-Basketball 3D

Technical Report and User Manual

Alessandro Carotenuto
MSc Artificial Intelligence & Robotics, Sapienza University of Rome

July 12, 2026

Interactive Graphics — Prof. Marco Schaerf
Dept. of Computer, Control and Management Engineering (DIAG)

Repository
https://github.com/SapienzaInteractiveGraphicsCourse/final-project-intentionallyblankname

Live demo
[TODO] GitHub Pages link, once deployed (see Section 10)

Contents

1 Project Overview

Mecha-Basketball 3D is a simulated basketball match between procedurally generated robots, built with Three.js and rendered on a GLTF basketball court. The user selects a robot class, each with a distinct stat profile and special move, then plays the match — steering, dribbling, aiming and shooting are all driven by real-time user input rather than a scripted replay.

No external 3D model is used for the robots: each class is built entirely in Three.js from primitives, organized in hierarchical `THREE.Group` structures and animated procedurally in JavaScript; the only two external assets are the court and the ball, both GLTF models (Section 2). Three robot classes are implemented, all fully playable by both the human player and the enemy AI, and swappable from the Main Menu without a page reload. Each class shares a homogeneous stat

struct `{ speed, shooting, steal, block }`, used both to drive gameplay (movement speed, shot-assist strength, steal/block cooldowns) and to render comparison bars on the selection screen.

Class	Locomotion	Special Move	Spd	Sht	Stl	Blk
Mobile Manipulator (AMR)	Wheels	Dash	3	1	3	3
Legged Manipulator	Legs (trot gait)	Jump	2	3	2	5
Drone	Flight (rotors)	Flight (grab-and-rise)	5	2	1	1

Table 1: Robot roster and stats (scale 1–5, except Shooting which is 1–3).

Two game modes are selectable from the Main Menu: **Practice** (one robot on an empty court) and **1v1** (an AI-controlled opponent with tactical navigation, defense, steal and block; first to 11 points wins).

2 Development Environment, Libraries and Assets

The project is built with **Three.js** (`^0.178.0`, npm-installed) on top of **Vite** (`^6.0.0`, dev server and bundler), in vanilla JavaScript ES modules — no TypeScript, no framework. Three.js was chosen over raw WebGL so the effort could go into modelling, animation and interaction rather than shader/buffer boilerplate. `vite.config.js` sets `base: './'` so the built bundle works from a GitHub Pages project subpath.

The renderer is configured for physically-plausible shading and soft shadows: `PCFSoftShadowMap` (VSM was tried and is too slow at 4096^2), ACES filmic tone mapping, sRGB output color space.

Beyond the Three.js core, the project uses only official Three.js *addons* (shipped with the `three` package itself): `GLTFLoader` (court and ball), `PointerLockControls` (mouse look), the `EffectComposer` post-processing stack (Section 4.2) and the procedural `Sky` skydome. No physics engine (Cannon.js, Ammo.js) was integrated: collision response and the dribble state machine are entirely hand-written. No animation library (e.g. tween.js, suggested by the course material) is used either: every animation — from a single joint rotation to the full shot trajectory — is computed by hand inside the render loop, satisfying the "no imported animations" requirement by construction rather than by omission.

Exactly two external, pre-made 3D assets are used:

- **Basketball court** (Sketchfab, modified): a custom `GLTFLoader` plugin re-implements the `KHR_materials_pbrSpecularGlossiness` extension (removed from Three.js r152+ with no maintained replacement); several material colors were hand-edited in the raw `scene.gltf` to remove banding; court-line z-fighting was fixed with `polygonOffset`.
- **Basketball**: a dedicated GLTF (not the low-quality sphere baked into the court model), with color, normal and metalness/roughness maps.

Every other visible object is procedurally generated: all three robot classes (geometry, materials and textures), the skybox, and all synthesized sound effects (Section 7). No robot model, texture image, audio file or animation clip was downloaded or imported.

3 Hierarchical Models

All three robot classes carry the same 3-revolute-joint manipulator arm on top of (or, for the Drone, hanging below) their locomotion base:

base (yaw) → link1 → elbow (pitch) → link2 (tapered) → wrist (pitch) → end effector + paddle

each joint being a nested `THREE.Group`, so that rotating a parent group (e.g. the elbow) carries every child (link2, wrist, paddle) with it — the textbook use of a scene-graph hierarchy for an articulated model. Two geometric details worth noting: `link2` uses a tapered (frustum)

cross-section built from a 4-sided `CylinderGeometry` (with `radialSegments` = 4 the cylinder degenerates into a square prism; different top/bottom radii turn it into a truncated pyramid); and the ball's resting point during handling is *not* the paddle's surface center but the convergence point of the two V-plates' normals, at distance $d/\cos(\alpha/2)$ from the wrist (α being the V opening angle) — using the surface center made the ball visibly interpenetrate the paddle.

Per-class hierarchy below the arm:

- **Mobile Manipulator**: 4 wheels (tori) grouped under a disc chassis; the wheel group's scale is derived from the disc radius, so resizing the chassis keeps the wheels attached.
- **Legged Manipulator**: 4 two-joint legs (hip + knee, both pitch) at the same offsets the wheels would occupy, each oriented radially outward so the legs radiate from the corners instead of all bending the same way; 25% larger overall. The flat foot is pivoted at its rear edge (ankle), not its center — the same off-center-pivot trick as the paddle.
- **Drone**: a body with four rotor arms (propellers spinning continuously, alternating direction between adjacent rotors); the 3R arm hangs *upside down* through a fixed flip group (`rotation.x` = π) inserted between the yaw base and the first link.

A thin OOP layer, `RobotBase`, wraps each procedural factory (copying its returned fields onto the instance with `Object.assign`), keeping geometry construction separated from game-entity code. It owns the shared `RobotState` FSM (`DRIBBLE` / `HANDLING` / `NO_BALL`), the stat-derived `speed` getter, and empty special-move hooks that each subclass overrides.

TODO: figure — Main Menu robot selection screen, 3 cards with live 3D preview

Figure 1: The robot selection screen, with live-dribbling 3D previews and stat bars.

4 Materials, Lighting and Textures

4.1 Lights

The scene uses 1 `HemisphereLight` (warm ambient term approximating the image-based lighting of the original Sketchfab scene), 1 `DirectionalLight` (key light with a 4096^2 shadow map, frustum sized to cover the whole court), 4 `PointLight` (court lamp posts, candela-scale intensity with physically-based inverse-square falloff) and 2 `SpotLight` (hoop fixtures, lit only at Sunset/Night — toggled via intensity, never `.visible`, to avoid a shadow-map-allocation stutter the first time a light becomes visible). All light colors, intensities and the sun position are driven by a `TimeOfDay` preset table (Sunrise/Day/Sunset/Night) and cross-fade smoothly (2.5 s, smoothstep-eased) between presets instead of snapping (Section 5).

Shadow tuning and scene scale. The court GLTF uses very large, roughly centimeter-scale units (thousands of units across), and every Three.js parameter with physical semantics had to be recalibrated, since library defaults assume scenes of ∼1–10 units: the shadow frustum must span ±2500 units, which at 4096^2 texels gives low texel density and caused shadow acne until `shadow.bias`/`normalBias` were tuned; point-light intensity is in candela with inverse-square decay, so at ∼250 units lamp-to-ground values around 2×10^5 are needed; and each lamp's light sits at the center of its solid glass globe, self-shadowing against its own shell unless `shadow.camera.near` is raised beyond the globe radius.

4.2 Post-Processing

Rendering goes through an `EffectComposer` pipeline — `RenderPass` → `SSAOPass` → `OutputPass` → `SMAAPass`. Two integration details are easy to get wrong. First, SSAO: `minDistance` / `maxDistance` are *normalized* fractions of the near→far depth range, not world units (world-scale

values silently disable the effect); a `camera.near` of 0.1 against a `far` of 5000 saturates depth-buffer precision (raised to 5); and the kernel radius had to shrink (90 → 12) to stop surfaces far behind a silhouette from reading as local occlusion (dark halos around edges). Second, `renderer.antialias` has no effect once rendering goes through the composer's internal render targets, so anti-aliasing is reintroduced with an `SMAAPass` as the last pass.

4.3 Textures

Two texture types are present, satisfying the "two different kinds" requirement with margin: **downloaded** (the basketball's color map and the court's diffuse color map) and **procedurally generated** — normal + metalness/roughness maps on the ball, and normal/roughness/bump maps generated at runtime (height-field → gradient) for every robot's materials, the wooden benches and the lamp posts.

4.4 Procedural Skybox

The flat background color was replaced with the `Sky` addon, a procedural atmospheric-scattering skydome (Preetham model) — no HDR image is downloaded. The sky's sun direction is always derived from the real `DirectionalLight` position (normalized), never tuned independently: an earlier version kept them separate and the sky's bright glow ended up 80–100° away from where shadows were cast; deriving both from one value makes them agree by construction. At Night the Preetham model degenerates (it is built for daylight scattering), so the sky mesh is swapped for a flat dark color once the animated transition has fully completed.

TODO: figure — close-up of a robot body showing procedural PBR texture detail

Figure 2: Procedurally generated normal/roughness detail on a robot's chassis.

5 Animation

Every animation is computed imperatively inside the render loop (`requestAnimationFrame`); nothing is imported, and no `AnimationMixer`/keyframe clip is used anywhere.

Why a fixed timestep. The dribble is where integrating on the variable render `delta` produced a real bug: the ball's release velocity was estimated as a single-frame finite difference, and whenever a frame hitch landed exactly on the frame where the push easing saturates, the estimated velocity collapsed to nearly zero — randomly — and the ball dropped dead instead of bouncing. Advancing the simulation in fixed 1/120 s steps through an accumulator (a slow frame runs several steps, a fast one may run none) removes the pathological case and makes the trajectory deterministic regardless of frame rate.

Third-person camera. While dribbling, the Play camera is a classic orbit camera (mouse yaw/pitch around the robot, `lookAt` pinned on it, pitch clamped to ∼3–51°); while handling the ball (aiming), it switches to a true free orientation — yaw/pitch drive `camera.quaternion` directly, decoupled from position — allowing to look *over* the robot toward the hoop, with an over-the-shoulder framing. Transitions between the two are always interpolated (position `lerp`, orientation quaternion `slerp`) toward a target recomputed every frame: the two formulas are structurally different, and switching without this layer produces a visible snap. The orbit yaw/pitch are deliberately *not* reset when entering handling (an earlier version reset the pitch, making the crosshair jump away from the aimed point); the handling pitch range is a strict superset of the dribbling one, so the aim direction stays continuous with no clamping.

System	Technique
Automatic dribble	Push/drop/rise finite-state machine at a fixed 120 Hz timestep, synchronized with the elbow/link1 joint motion
Shot flight	Projectile motion under constant gravity, sub-stepped at 240 Hz (anti-tunnelling), with full collision response
Wheel steering	Exponential-smoothing angular interpolation (shortest-path, wrap-safe) toward the movement direction
Legged gait	Diagonal-pair trot cycle, advancing only while the robot's position actually changes frame-to-frame
Drone flight feel	Body bank (turning) and thrust-tilt (accelerating), composed via explicit quaternion multiplication rather than Euler angles, so the tilt axis stays attached to the drone's nose at any yaw
Special moves	Dash (6×-speed burst, 4 s cooldown), Jump (parabolic arc, 5 s cooldown, only without the ball), Flight (grab→rise→hold→descend, rigid ball attachment throughout)
Shot animation	Windup/release/recover arm pose, elbow coupled to camera pitch while aiming
Time-of-Day fades	2.5 s smoothstep cross-fade of every light/sky parameter, running every frame in any mode
Ball spin	Derived each frame from real displacement (axis $= \hat{u} \times \vec{v}$, rate $= \lvert\vec{v}\rvert / r$), so spin is consistent across dribble/shot/handling/pickup with no per-state simulation

Table 2: Animated systems (non-exhaustive; static geometry is limited to the court and lamp posts).

6 Physics, Collisions and Scoring

No physics engine is used; every collision test is a hand-written closed-form formula: **sphere vs. AABB** (ball vs. backboard/walls/poles/benches, and robot vs. static court geometry, so a Dash cannot push a robot through a wall) and **sphere vs. torus** (ball vs. rim). Both use the same reflection formula $\vec{v}' = \vec{v} - (1 + e)(\vec{v} \cdot \hat{n})\hat{n}$, with a per-material restitution e (backboard 0.15, rim 0.3, walls/poles 0.55, benches 0.5). The 66 wall/bleacher collision boxes are extracted mesh-by-mesh from the real court GLTF (not one aggregate box), since the venue has intentional gaps between panels that an aggregate would misrepresent.

Two robustness details: shot flight is sub-stepped at 240 Hz because at full shot speed a single frame can move the ball farther than the backboard is thick (tunnelling); and post-bounce collision cooldowns are tracked *per object*, because a single global cooldown after a rim bounce would suspend the backboard check at exactly the moment the deflected ball reaches it.

A "hoop assist" potential field gently pulls an airborne shot toward the rim center, with strength scaled by the shooter's `shooting` stat. It is a clamped *position* correction rather than an acceleration: an acceleration accumulates with time spent in the assist cone, so slow close-range shots were flung *past* the rim — a position pull clamped at the center cannot overshoot by construction.

Scoring: 2 points if the shooter was inside the three-point arc *at the moment of release*, 3 otherwise; the arc radius is read from the court GLTF's real accessor data. A basket is detected by interpolating the exact crossing point of the rim's plane between two consecutive physics samples — testing only the sample already past the plane misses steep, close-range shots that genuinely went in.

7 Audio

All five sound effects are synthesized at runtime with the Web Audio API — no audio file is shipped: **score** (two ascending sine notes, a fifth apart), **bounce** (a low downward-sweeping tone plus a short low-pass noise transient), **shoot** (band-pass filtered white noise sweeping upward — a rising "whoosh"), **steal** (the same band-pass technique sweeping *downward*, much shorter —

a "swipe"), and **block** (low-pass noise hit over a descending low tone). Each has a short linear attack before its exponential decay — an instantaneous jump to peak gain produces an audible click. A background music loop is not yet implemented (Section 10).

8 User Interaction and Controls

The Main Menu (pure HTML/CSS overlaid on the canvas) walks through Gamemodes → Robot selection (a second "Robot Opponent" screen appears only in 1v1) → Time of Day + Start. All three robot classes are instantiated (hidden) at page load, so switching class is a visibility toggle, never a page reload. The selection cards run the *same* dribble state machine as the real game on a live offscreen renderer — the preview robot genuinely dribbles, no pre-rendered screenshot.

Input	Effect
`P`	Toggle the debug panel (per-component geometry tuning + live camera readout)
`M`	Toggle Spectate (free-fly) / Play (third-person) mode
Mouse (Play)	Orbits the camera around the robot; aims the arm
`WASD` (Play)	Move relative to camera facing; wheels/legs steer toward the movement direction
Left Shift (Play)	Special move for the active class (Dash / Jump / Flight)
Right mouse, held (Play)	`HANDLING` state — pauses the dribble, frees the camera to aim toward the hoop
Left click (`HANDLING`)	Shoot; direction from a raycast through the crosshair, constant launch speed, reduced to 60% inside the three-point arc
Walk near a loose ball	Automatic pickup, no key required
`Q` / `E` (1v1, no ball)	Steal (sweep toward the opponent) / Block (deflects a shot in flight)
`Esc` (Play)	Pause menu
`1-8`	Toggle collision/contact wireframe overlays (backboard, rim, walls, poles, benches, steal zone, block radius, pickup zone)

Table 3: Full control scheme.

Shot direction comes from a raycast through the crosshair pixel, and shot *strength* is deliberately constant rather than charge-based: varying the force at aim time would change the previewed trajectory while lining it up. Instead, a live trajectory preview (a 3D tube rebuilt every frame from the same physics as the real flight, collisions included) shows exactly where the shot will go, turning green when the simulated path crosses the hoop.

Beyond gameplay controls, the debug panel (key `P`) exposes every tunable parameter — robot geometry and gameplay timing constants — as live sliders, plus a "Copy Config" button that serializes the current geometry to the clipboard: the shipped default dimensions of all three robots were tuned visually this way and pasted back as code. An "Animation Preview" section can force any pose (Dribble/Handling/Shoot/Special Move) for inspection from a free camera angle.

TODO: figure — debug panel open, showing per-component sliders

Figure 3: The debug panel, used to tune every robot's geometry live.

9 1v1 Mode and Enemy AI

The opponent is driven by a small tactical finite-state machine, `EnemyState` ∈ {`CHASE_BALL`, `ATTACK`, `DEFEND`}, recomputed every frame from who currently possesses the ball: it chases a loose ball, attacks toward its hoop and shoots once in range, and otherwise positions itself between the

player and the player's hoop while attempting a Steal or Block. AI shot elevation is solved with the standard ballistic-projectile equation at fixed launch speed, falling back to a 50° arc if the target is unreachable.

STEAL and BLOCK are shared between player and AI through a single module: a reach/sweep animation, a real bounding-box contact test (asymmetric for Steal — wide margin in front of the sweeping robot, almost none behind, so a Steal cannot succeed from behind), and a resolve step that re-validates possession *at the end* of the sweep rather than at first contact. Robot-vs-robot overlap is resolved with a possession-based asymmetric position correction (the ball holder yields 25% of the separation, the other robot 75%); a robot mid-shot is never displaced at all, because its position also drives the chase camera and being nudged during release made the crosshair slide at the exact moment of the shot.

10 Conclusions and Limitations

Table 4 maps the course's mandatory requirements onto the sections of this report.

Requirement	**Where it is satisfied**	**Section**
Hierarchical model with structural animation	Three procedural robots, each a nested-`Group` hierarchy (locomotion base → 3R arm → paddle); steering, aiming, dribbling and the special moves all animate *through* the hierarchy	§3
At least one light	1 hemisphere + 1 directional (shadowed) + 4 point + 2 spot, all driven by the Time-of-Day presets	§4
Textures of two different kinds	Ball: color + normal + metalness/roughness maps; robots/benches/lamps: procedurally generated normal/roughness/bump maps	§4
Real user interaction	Full match controls (move/aim/dribble/shoot/steal/block), Main Menu with robot/game-mode/time-of-day selection, pause and options	§8
Most objects animated, all in JavaScript	Every robot, the ball, the shot, the sky and the lights are animated imperatively in the render loop; nothing imported	§5

Table 4: Coverage of the course's mandatory requirements.

Beyond the requirements, the two choices that shaped the project most were building *everything* procedurally (robots, textures, skybox, sounds — only the court and ball are external) and simulating on fixed timesteps decoupled from the render rate, which made both the dribble and the shot deterministic and debuggable. The recurring engineering lesson, met independently in the shadow setup, the SSAO configuration and the light intensities, is that a graphics library's physically-motivated defaults silently assume a scene scale — importing an asset with different units means re-deriving every such parameter rather than tuning it blindly.

Known limitations: there is no 3v3 mode; there is no background music (only the synthesized SFX of Section 7); robot color customization intentionally does not persist across a page refresh (in-memory, session only).

- **[TODO]** GitHub Pages deployment — `base: './'` is already configured, but the project is not deployed yet; once it is, fill in the live link on the title page and in the repository's `README.md`.
- **[TODO]** personal conclusions / retrospective on the project, in the student's own words.